

```
In [1]: import pandas as pd

In [2]: import numpy as np

In [3]: import matplotlib.pyplot as plt

In [4]: import seaborn as sns

In [5]: import seaborn as sns

In [6]: from sklearn.model_selection import train_test_split

In [7]: from sklearn.preprocessing import LabelEncoder, OneHotEncoder, StandardScaler

In [8]: from sklearn.linear_model import LinearRegression

In [9]: from sklearn.ensemble import RandomForestRegressor

In [10]: from sklearn.metrics import mean_squared_error, r2_score

In [11]: import zipfile

In [12]: zip_path = "IMDb Movies India.csv.zip"
with zipfile.ZipFile(zip_path, 'r') as z:
    z.extractall("data")

In [13]: import os

In [14]: print(os.listdir("data"))

['IMDb Movies India.csv']

In [15]: df = pd.read_csv("data/IMDb Movies India.csv", encoding="utf-8")
```

```

-----
UnicodeDecodeError                                Traceback (most recent call last)
File parsers.pyx:1120, in pandas._libs.parsers.TextReader._convert_tokens()

File parsers.pyx:1272, in pandas._libs.parsers.TextReader._convert_with_dtype()

File parsers.pyx:1285, in pandas._libs.parsers.TextReader._string_convert()

File parsers.pyx:1535, in pandas._libs.parsers._string_box_utf8()

UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe9 in position 3: invalid continuation byte

During handling of the above exception, another exception occurred:

UnicodeDecodeError                                Traceback (most recent call last)
Cell In[15], line 1
----> 1 df = pd.read_csv("data/IMDb Movies India.csv", encoding="utf-8")

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\io\parsers\readers.py:1026, in read_csv(filepath_or_buffer, sep, delimiter, header, names, index_col, usecols, dtype, engine, converters, true_values, false_values, skip_initialspace, skiprows, skipfooter, nrows, na_values, keep_default_na, na_filter, verbose, skip_blank_lines, parse_dates, infer_datetime_format, keep_date_col, date_parser, date_format, dayfirst, cache_dates, iterator, chunk_size, compression, thousands, decimal, lineterminator, quotechar, quoting, doublequote, escapechar, comment, encoding, encoding_errors, dialect, on_bad_lines, delim_whitespace, low_memory, memory_map, float_precision, storage_options, dtype_backend)
    1013 kwds_defaults = _refine_defaults_read(
    1014     dialect,
    1015     delimiter,
    (...)
    1022     dtype_backend=dtype_backend,
    1023 )
    1024 kwds.update(kwds_defaults)
-> 1026 return _read(filepath_or_buffer, kwds)

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\io\parsers\readers.py:626, in _read(filepath_or_buffer, kwds)
    623     return parser
    625 with parser:
--> 626     return parser.read(nrows)

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\io\parsers\readers.py:1923, in TextFileReader.read(self, nrows)
    1916 nrows = validate_integer("nrows", nrows)
    1917 try:
    1918     # error: "ParserBase" has no attribute "read"
    1919     (
    1920         index,
    1921         columns,
    1922         col_dict,
-> 1923     ) = self._engine.read( # type: ignore[attr-defined]
    1924         nrows
    1925     )
    1926 except Exception:
    1927     self.close()

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\io\parsers\c_parser_wrapper.py:234, in CParserWrapper.read(self, nrows)
    232 try:
    233     if self.low_memory:
--> 234         chunks = self._reader.read_low_memory(nrows)
    235         # destructive to chunks
    236         data = _concatenate_chunks(chunks)

File parsers.pyx:838, in pandas._libs.parsers.TextReader.read_low_memory()

File parsers.pyx:921, in pandas._libs.parsers.TextReader._read_rows()

File parsers.pyx:1066, in pandas._libs.parsers.TextReader._convert_column_data()

File parsers.pyx:1127, in pandas._libs.parsers.TextReader._convert_tokens()

File parsers.pyx:1272, in pandas._libs.parsers.TextReader._convert_with_dtype()

File parsers.pyx:1285, in pandas._libs.parsers.TextReader._string_convert()

File parsers.pyx:1535, in pandas._libs.parsers._string_box_utf8()

UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe9 in position 3: invalid continuation byte

```

```
In [16]: df = pd.read_csv("data/IMDb Movies India.csv", encoding="latin1")
```

```
In [17]: print(df.head())
```

	Name	Year	Duration	Genre	\
0		NaN	NaN		Drama
1	#Gadhvi (He thought he was Gandhi)	(2019)	109 min		Drama
2	#Homecoming	(2021)	90 min	Drama, Musical	
3	#Yaaram	(2019)	110 min	Comedy, Romance	
4	...And Once Again	(2010)	105 min		Drama

	Rating	Votes	Director	Actor 1	Actor 2	\
0	NaN	NaN	J.S. Randhawa	Manmauji	Birbal	
1	7.0	8	Gaurav Bakshi	Rasika Dugal	Vivek Ghamande	
2	NaN	NaN	Soumyajit Majumdar	Sayani Gupta	Plabita Borthakur	
3	4.4	35	Ovais Khan	Prateik	Ishita Raj	
4	NaN	NaN	Amol Palekar	Rajat Kapoor	Rituparna Sengupta	

	Actor 3
0	Rajendra Bhatia
1	Arvind Jangid
2	Roy Angana
3	Siddhant Kapoor
4	Antara Mali

In [18]: `print(df.info())`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15509 entries, 0 to 15508
Data columns (total 10 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Name         15509 non-null  object
1   Year         14981 non-null  object
2   Duration     7240 non-null   object
3   Genre        13632 non-null  object
4   Rating       7919 non-null   float64
5   Votes        7920 non-null   object
6   Director     14984 non-null  object
7   Actor 1     13892 non-null  object
8   Actor 2     13125 non-null  object
9   Actor 3     12365 non-null  object
dtypes: float64(1), object(9)
memory usage: 1.2+ MB
None
```

In [19]: `print(df.describe())`

	Rating
count	7919.000000
mean	5.841621
std	1.381777
min	1.100000
25%	4.900000
50%	6.000000
75%	6.800000
max	10.000000

In [20]: `print(df.isnull().sum())`

Name	0
Year	528
Duration	8269
Genre	1877
Rating	7590
Votes	7589
Director	525
Actor 1	1617
Actor 2	2384
Actor 3	3144
dtype:	int64

In [21]: `df = df.dropna(subset=["IMDb Rating"])`

```
-----
KeyError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_9752\1709795256.py in ?()
----> 1 df = df.dropna(subset=["IMDb Rating"])

C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\frame.py in ?(self, axis, how, thresh, subset, inplace, ignore_index)
    6666         ax = self._get_axis(agg_axis)
    6667         indices = ax.get_indexer_for(subset)
    6668         check = indices == -1
    6669         if check.any():
-> 6670             raise KeyError(np.array(subset)[check].tolist())
    6671         agg_obj = self.take(indices, axis=agg_axis)
    6672
    6673         if thresh is not lib.no_default:

KeyError: ['IMDb Rating']
```

```
In [22]: print(df.columns)
```

```
Index(['Name', 'Year', 'Duration', 'Genre', 'Rating', 'Votes', 'Director',
       'Actor 1', 'Actor 2', 'Actor 3'],
      dtype='object')
```

```
In [23]: df = df.dropna(subset=["IMDb_Rating"])
```

```
-----
KeyError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_9752\2762113639.py in ?()
----> 1 df = df.dropna(subset=["IMDb_Rating"])

C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\frame.py in ?(self, axis, how, thresh, subset, inplace, ignore_index)
    6666         ax = self._get_axis(agg_axis)
    6667         indices = ax.get_indexer_for(subset)
    6668         check = indices == -1
    6669         if check.any():
-> 6670             raise KeyError(np.array(subset)[check].tolist())
    6671         agg_obj = self.take(indices, axis=agg_axis)
    6672
    6673         if thresh is not lib.no_default:

KeyError: ['IMDb_Rating']
```

```
In [24]: print(df.columns.tolist())
```

```
['Name', 'Year', 'Duration', 'Genre', 'Rating', 'Votes', 'Director', 'Actor 1', 'Actor 2', 'Actor 3']
```

```
In [25]: df = df.dropna(subset=["Rating"])
```

```
In [26]: df["Genre"] = df["Genre"].fillna("Unknown")
```

```
C:\Users\neha gupta\AppData\Local\Temp\ipykernel_9752\3663634611.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
df["Genre"] = df["Genre"].fillna("Unknown")
```

```
In [27]: df = df.copy()
```

```
In [28]: df["Genre"] = df["Genre"].fillna("Unknown")
```

```
In [29]: df["Director"] = df["Director"].fillna("Unknown")
```

```
In [30]: df["Actor 1"] = df["Actor 1"].fillna("Unknown")
```

```
In [31]: df["Actor 2"] = df["Actor 2"].fillna("Unknown")
```

```
In [32]: df["Actor 3"] = df["Actor 3"].fillna("Unknown")
```

```
In [33]: label_enc = LabelEncoder()
```

```
In [34]: df["Genre"] = label_enc.fit_transform(df["Genre"])
```

```
In [35]: df["Director"] = label_enc.fit_transform(df["Director"])
```

```
In [36]: df["Actors"] = label_enc.fit_transform(df["Actors"])
```

```

-----
KeyError                                Traceback (most recent call last)
File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:3805, in Index.get_loc(self, key)
    3804 try:
-> 3805     return self._engine.get_loc(casted_key)
    3806 except KeyError as err:

File index.pyx:167, in pandas._libs.index.IndexEngine.get_loc()

File index.pyx:196, in pandas._libs.index.IndexEngine.get_loc()

File pandas\_libs\hashtable_class_helper.pxi:7081, in pandas._libs.hashtable.PyObjectHashTable.get_item()

File pandas\_libs\hashtable_class_helper.pxi:7089, in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'Actors'

The above exception was the direct cause of the following exception:

KeyError                                Traceback (most recent call last)
Cell In[36], line 1
----> 1 df["Actors"] = label_enc.fit_transform(df["Actors"])

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\frame.py:4102, in DataFrame.__getitem__(self, key)
    4100 if self.columns.nlevels > 1:
    4101     return self._getitem_multilevel(key)
-> 4102 indexer = self.columns.get_loc(key)
    4103 if is_integer(indexer):
    4104     indexer = [indexer]

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:3812, in Index.get_loc(self, key)
    3807 if isinstance(casted_key, slice) or (
    3808     isinstance(casted_key, abc.Iterable)
    3809     and any(isinstance(x, slice) for x in casted_key)
    3810 ):
    3811     raise InvalidIndexError(key)
-> 3812     raise KeyError(key) from err
    3813 except TypeError:
    3814     # If we have a listlike key, _check_indexing_error will raise
    3815     # InvalidIndexError. Otherwise we fall through and re-raise
    3816     # the TypeError.
    3817     self._check_indexing_error(key)

KeyError: 'Actors'

```

```
In [37]: df["Actor 1"] = label_enc.fit_transform(df["Actor 1"])
```

```
In [38]: df["Actor 2"] = label_enc.fit_transform(df["Actor 2"])
```

```
In [39]: df["Actor 3"] = label_enc.fit_transform(df["Actor 3"])
```

```
In [40]: X = df[["Genre", "Director", "Actor 1", "Actor 2", "Actor 3"]]
```

```
In [41]: y = df["Rating"]
```

```
In [42]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
In [43]: lr = LinearRegression()
```

```
In [44]: lr.fit(X_train, y_train)
```

```
Out[44]: ▼ LinearRegression ⓘ ?
LinearRegression()
```

```
In [45]: y_pred_lr = lr.predict(X_test)
```

```
In [46]: rf = RandomForestRegressor(n_estimators=100, random_state=42)
```

```
In [47]: rf.fit(X_train, y_train)
```

```
Out[47]: ▼ RandomForestRegressor ⓘ ?
RandomForestRegressor(random_state=42)
```

```
In [48]: y_pred_rf = rf.predict(X_test)
```

```
In [49]: print(" Linear Regression Performance:")
```

Linear Regression Performance:

```
In [50]: print("R2 Score:", r2_score(y_test, y_pred_lr))
```

R2 Score: 0.035450225863035256

```
In [51]: print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_lr)))
```

RMSE: 1.3391183762067467

```
In [52]: print("\n Random Forest Performance:")
```

Random Forest Performance:

```
In [53]: print("R2 Score:", r2_score(y_test, y_pred_rf))
```

R2 Score: 0.11379742465064835

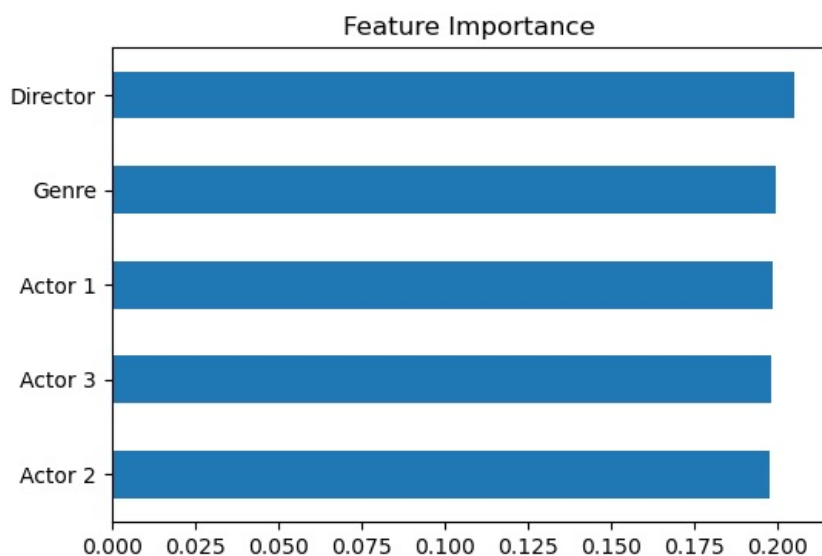
```
In [54]: print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_rf)))
```

RMSE: 1.2835806213813172

```
In [55]: feat_importance = pd.Series(rf.feature_importances_, index=X.columns)
```

```
In [56]: feat_importance.sort_values().plot(kind="barh", title="Feature Importance", figsize=(6,4))
```

Out[56]: <Axes: title={'center': 'Feature Importance'}>



```
In [57]: plt.show()
```

```
In [ ]:
```